

Spark study

Spark 개념 정리

Scala vs Python 성능 비교

일반적인 성능

- **Scala**: 성능 우수 (JVM 네이티브)
- **Python**: 개발 속도 & 에코시스템 우월

UDF (User Defined Functions)

복잡한 UDF는 **Scala**가 압도적으로 유리

Python UDF의 단점

- 각 row마다 Python process와 통신 필요
- JVM ↔ Python 직렬화/역직렬화 오버헤드

해결 방법

Scala로 작성한 UDF를 Python에서 사용

- Scala 코드를 **JAR로 컴파일**
- Spark 클러스터에 등록
- Python에서 호출 → 큰 병목 없음

Catalyst Optimizer

쿼리를 분석하고 최적화하는 Spark의 핵심 엔진

1 Analysis (분석)

테이블/컬럼이 실제로 존재하는지 확인

- 데이터 타입 체크

- 카탈로그(메타데이터)와 대조
- Unresolved Logical Plan → Resolved Logical Plan 변환

2 Logical Optimization (논리적 최적화)

쿼리를 논리적으로 더 효율적으로 변환

최적화 기법	설명
Predicate Pushdown	필터를 가능한 빨리 적용
Projection Pruning	필요한 컬럼만 호출
Constant Folding	상수 계산 미리 수행
Boolean Expression Simplification	조건 단순화

3 Physical Planning (물리적 계획)

논리 계획을 실제 실행 계획으로 변환

CBO 기반 Join 전략 선택

1. Broadcast Hash Join - 작은 테이블 브로드캐스트
2. Sort Merge Join - 큰 테이블 간 조인
3. Shuffle Hash Join - 중간 크기 테이블

4 Code Generation (코드 생성)

최적화된 Java 바이트코드 생성

Whole-Stage Code Generation (Tungsten Project)

- 여러 operator를 하나의 함수로 합쳐서 생성
- Virtual function call 오버헤드 제거
- CPU 캐시 활용 극대화

Spark 실행 구조

```

Application (전체 프로그램)
└─ Job (Action 호출 시 생성)
  
```

- └ Stage (Shuffle 경계로 분리)
- └ Task (파티션별 실행)

Action의 종류

Note

1개의 Action = 1개의 Job

```
show() · collect() · count() · take() · first() · head()  
write.parquet() · save() · foreach() · reduce()
```

Stage

Shuffle 경계로 나뉘는 실행 단위

Shuffle이란?

데이터를 네트워크를 통해 재분배하는 작업

Narrow vs Wide Transformation

● Narrow Transformation

- 연산: `map`, `filter`, `select`, `union`
- 특징: 각 파티션 독립적으로 처리 가능
- Shuffle: 발생 안 함

● Wide Transformation

- 연산: `groupBy`, `join`, `repartition`, `sortBy`
- 특징: 여러 파티션의 데이터 조합 필요
- Shuffle: 발생! (Stage 분리)

Task

각 파티션을 처리하는 가장 작은 실행 단위

핵심 개념

- Stage 0에 10개 파티션 → 10개 Task 생성
- 하나의 Executor Core에서 하나의 Task 실행
- 병렬 실행 가능 Task 수 = 사용 가능한 Core 수

예시

Executor: 4 cores × 10개 = 총 40 cores

파티션: 200개

→ 200개 Task를 40개씩 5번에 걸쳐 실행

Hadoop + Spark

Storage(HDFS)

- 노드 추가로 쉽게 확장
- PB 단위 데이터 저장
- 비용이 저렴 (?)

리소스 관리(Yarn)

- 멀티테넌시 - 하나의 소프트웨어 인스턴스나 애플리케이션이 여러 고객을 동시에 서비스 하는 아키텍처 패턴
- 리소스 격리
- 우선순위(Job 우선순위 설정 가능)
- 큐 관리

Data Locality

- 데이터를 네트워크로 옮기는 것보다, task를 데이터가 있는 곳으로 보내는 게 훨씬 빠름.
- Spark 가 데이터가 있는 노드에서 task 실행 (shuffle 최소화)

1. Data Locality level

1. **Process_Local**: 같은 JVM process, 실행단위
2. **Node_Local**: 같은노드, 물리적 서버 1대 단위
3. **Rack_Local**: 같은 랙, 노드 단위(1개의 랙에 10 개의 노드)
4. **ANY**: 다른랙

Spark Lazy Evaluation

Lazy Evaluation

| 지연연산, 필요시까지 실행을 미룸

- Transformation 에서 계획만 세우고 Action 에서 실제로 실행하는 것
- 지연연산으로 전체적인 최적화를 할 수 있음.

query push down

| 연산을 데이터 소스(DB, parquet 등) 밀어내려서 데이터를 적게 가져오는 최적화 기법

예시

`df = spark.read.parquet("users.parquet")` # users 테이블: id, name, age, email, address, phone, created_at (7개 컬럼)

```
df_selected = df.select("name", "age") # 2개만 필요 df_selected.show()
```

- DB 에서 가져온다고 봤을 때, 단순히 DB 에 부하를 줄 것인가 app 에 부하를 줄 것인가? 에 대한 것

cluster mode

- Driver 란 ?
 - main 함수 실행
 - SparkSession 실행
 - Job 스케줄링
 - Task 분배
- Executor 란?
 - 실제 데이터 처리
 - Task 실행
 - 결과반환

cluster mode

| 드라이버가 클러스터 안에 있는 구조

장점

- 실시간 로그 확인 가능
- 디버깅이 쉬움
- jupyter notebook, IDE 에서 사용

단점

- 네트워크 트래픽 많음
- 클라이언트 꺼지면 중단

결론

대화형 작업, 개발/테스트, 실시간 모니터링의 경우 사용

client mode

| 드라이버가 클라이언트에 있는 구조

장점

- 클라이언트 꺼져도 Job 계속 실행
- 네트워크 트래픽 적음
- Prod 환경 적합

단점

- 실시간 로그 확인은 Spark UI
- 디버깅 어려움
- 대화형 작업 X

결론

운영계 전용

Spark context, session

| context 는 과거 버전으로 객체 - 클러스터 연결
session 은 현재 버전으로 모든 spark 기능의 시작점

context

- 싱글톤 패턴: JVM 당 1개만 존재가능
- 저수준 API
- RDD 생성 및 관리
- 브로드캐스트 변수(작은 읽기 전용 데이터를 모든 executor 에 복사)
- accumulator(누산기, 여러 Task 의 결과를 안전하게 합산 - action 에서만)

session

- Spark 2.X 버전 이후 권장 방식
- spark context 를 내부에 포함
- 고수준 API
- DF/dataset, SQL , hive, Streaming 통합

결론

context 를 아예 안쓰는 것이 아니라, RDD 나 broadcast 변수 등이 필요하면 사용함. 그 외에는 spark session 이 맞음.

Tungsten

Spark 의 실행 엔진을 CPU 와 메모리 레벨에서 최적화하는 프로젝트

핵심기술

- Memory Management
 - off-heap Memory
 - Binary 형태로 저장(<-> 디버깅 어려움)
 - 직접 메모리관리(GC 무관 <-> 수동 메모리 관리 및 누수위험)
 - 오버헤드 최소화
 - 메모리 연속 배치
 - Memory 레이아웃 최적화
- Whole-Stage Code Generation(코드 생성)
 - 코드 최적화
- Cashe-Aware Computation(캐시 최적화)
 - cpu cash 활용
 - SIMD 명령어 활용 가능
 - single instruction, multiple data
 - 하나의 cpu 명령어로 여러 데이터를 동시에 처리하는 기술
 - 압축 효율 증가
- Binary Processing(이진 처리, 직렬화 제거)

- Tungsten 이전에는 네트워크 전송시에 직렬화/역직렬화 단계가 있었으나, 이후에는 없음.

결론

spark 2.x 이후에 자동 적용되는 최적화 엔진이나, 성능 튜닝관점이나, 디버깅 제약, spark 가 왜 빠른가? 에 대해 깊게 이해할 수 있음.

포톤엔진

databricks가 개발한 차세대 네이티브 벡터화 쿼리 엔진

핵심개념

- Tungsten 은 Scala/JVM 으로 구현된 반면에 Photon 은 C++ 로 구현
- Tungsten 은 JVM 기반 최적화, Photon은 JVM 없이 CPU 직접 실행
- SIMD 최적화 강화: 적극적 SIMD, 최신 CPU 명령어
- 적응형 실행: 런타임 통계 기반 최적화, 동적 파티션 조정
- Apache Spark 불가능
-